Big Data A2 — Simple Search Engine using Hadoop MapReduce

Course: Big Data — IU

Student: Ksenia Korchagina AI-01

1. Methodology

1.1 Overview

The search engine is built as a fully automated pipeline with four stages:

1. **Data preparation** — PySpark reads plain-text documents and builds a structured input file in HDFS.
2. **Indexing** — two Hadoop MapReduce Streaming pipelines produce an inverted index and document statistics, stored in HDFS.
3. **Index loading** — a Python script reads the HDFS output and loads it into four Cassandra tables.
4. **Query answering** — a PySpark application runs on YARN, reads the Cassandra index, computes BM25 scores via the RDD API, and returns the top-10 ranked documents.

The pipeline is orchestrated by `app.sh`, set as the container entry point in `docker-compose.yml`. Running `docker compose up` reproduces the complete system from scratch.

1.2 Data Preparation

The corpus consists of **1,000 Wikipedia articles** from the `m.parquet` file (Wikipedia 20230701 dataset). Each article is stored as a plain-text file named `<doc_id>_<doc_title>.txt`, with spaces replaced by underscores. The files are committed to `app/data/` in the repository, so no parquet download is needed at runtime.

All documents are uploaded to HDFS `/data` with a single `hdfs dfs -put` call launched from Python. Files with non-ASCII characters in their names are filtered out before the call to prevent the HDFS JVM client from returning a non-zero exit code, which would abort the pipeline. The number of skipped files is logged as a warning.

`prepare_input.py` then reads HDFS `/data` with `sc.wholeTextFiles()`, parses the `doc_id` and `doc_title` from each filename, and produces one tab-separated line per document (`doc_id\\ttitle\\ttext`). The RDD is coalesced to one partition and saved to HDFS `/input/data`, which is the input format expected by the MapReduce indexer.

1.3 Indexer (Hadoop MapReduce Streaming)

Two pipelines are run by `create_index.sh` using the Hadoop Streaming JAR. Both read from `/input/data` with `mapreduce.job.reduces=1` to produce single-partition output.

Pipeline 1 — Inverted Index (`mapper1.py` + `reducer1.py`)

The mapper tokenises each document's text with the regex `[a-z]{2,}` and emits `term\\tdoc_id` for every token. The reducer receives pairs sorted by term, accumulates per-document term frequencies, and emits one line per term:

```
<term>\\t<df>\\t<doc_id1>:<tf1> <doc_id2>:<tf2> …
```

Storing the full posting list on one line reduces the number of Cassandra write operations and makes the output easy to inspect. Document frequency is computed as `len(postings)` directly in the reducer, avoiding an extra MapReduce pass.

Pipeline 2 — Document and Corpus Statistics (`mapper2.py` + `reducer2.py`)

The mapper tokenises each document and emits `__STATS__\\tdoc_id\\ttitle\\tdoc_length`. The sentinel key `__STATS__` routes all records to the single reducer, which computes the average document length globally and writes one `DOC` line per document and one `CORPUS` line with the corpus-wide totals.

1.4 Cassandra Schema

`load_to_cassandra.py` loads both HDFS outputs into the `search_index` keyspace:

Table	Primary Key	Columns	Purpose
`vocabulary`	`term`	`df INT`	IDF computation
`inverted_index`	`(term, doc_id)`	`tf INT`	TF lookup per posting
`doc_stats`	`doc_id`	`title TEXT`, `doc_length INT`	BM25 length normalisation
`corpus_stats`	`id INT` (always 1)	`total_docs INT`, `avg_doc_length DOUBLE`	Global BM25 parameters

`inverted_index` uses a composite primary key `(term, doc_id)` so all postings for a term are fetched with a single `WHERE term = ?` query. `vocabulary` is kept separate to allow IDF to be computed before loading full posting lists. Tables are truncated before each re-index to guarantee a clean state. Bulk writes use `execute_concurrent_with_args` with concurrency 64.

1.5 BM25 Ranker (`query.py` + `search.sh`)

`query.py` tokenises the query with the same regex as the indexer, then fetches from Cassandra on the driver: corpus stats, per-term `df` and posting lists, and per-document lengths. This happens before the Spark context is created, which is efficient for a 1,000-document corpus where posting lists fit in driver memory.

The posting data is parallelised into an RDD. `flatMap` applies the BM25 formula per term:

$$\mathrm{BM25}(t,d) = \mathrm{IDF}(t) \cdot \frac{tf(t,d),(k_1+1)}{tf(t,d) + k_1\left(1 - b + b, \frac{|d|}{\mathrm{avgdl}}\right)}$$

with `k1 = 1.5` and `b = 0.75`. `avgdl` and document metadata are broadcast to executors. `reduceByKey` sums scores across query terms, and `sortBy(...).take(10)` returns the top-10 results.

`search.sh` submits the job to YARN in client mode, distributing the Python virtualenv via `--archives` and pointing to a pre-uploaded `spark-jars.zip` in HDFS to avoid localising hundreds of individual JARs per job.

2. Demonstration

2.1 How to Run

Requirements: Docker and Docker Compose. No other local dependencies.

```
git clone <repository-url>
cd BigDataA2
docker compose up
```

`docker compose up` streams all container logs to the terminal. `cluster-master` automatically runs `app.sh` and executes the full pipeline. On first run this takes approximately **25–35 minutes** (the slowest step is packaging and uploading ~300 MB of Spark JARs to HDFS, which is cached on subsequent runs).

Progress is shown in the logs:

```
[Step 1/8] Starting SSH server...
[Step 2/8] Starting Hadoop / YARN / Spark services...
[Step 3/8] Setting up Python virtualenv...
[Step 4/8] Uploading documents to HDFS /data...
[Step 5/8] Building /input/data with PySpark...
[Step 6-7/8] Building index and loading to Cassandra...
```

```
[Step 8/8] Running test search query...
Pipeline complete! Container is running.
```

Once the pipeline is complete, run queries from a separate terminal:

```
docker exec cluster-master bash /app/search.sh "machine learning"
docker exec cluster-master bash /app/search.sh "military aircraft"
docker exec cluster-master bash /app/search.sh "computer science algorithm"
```

2.2 Successful Indexing

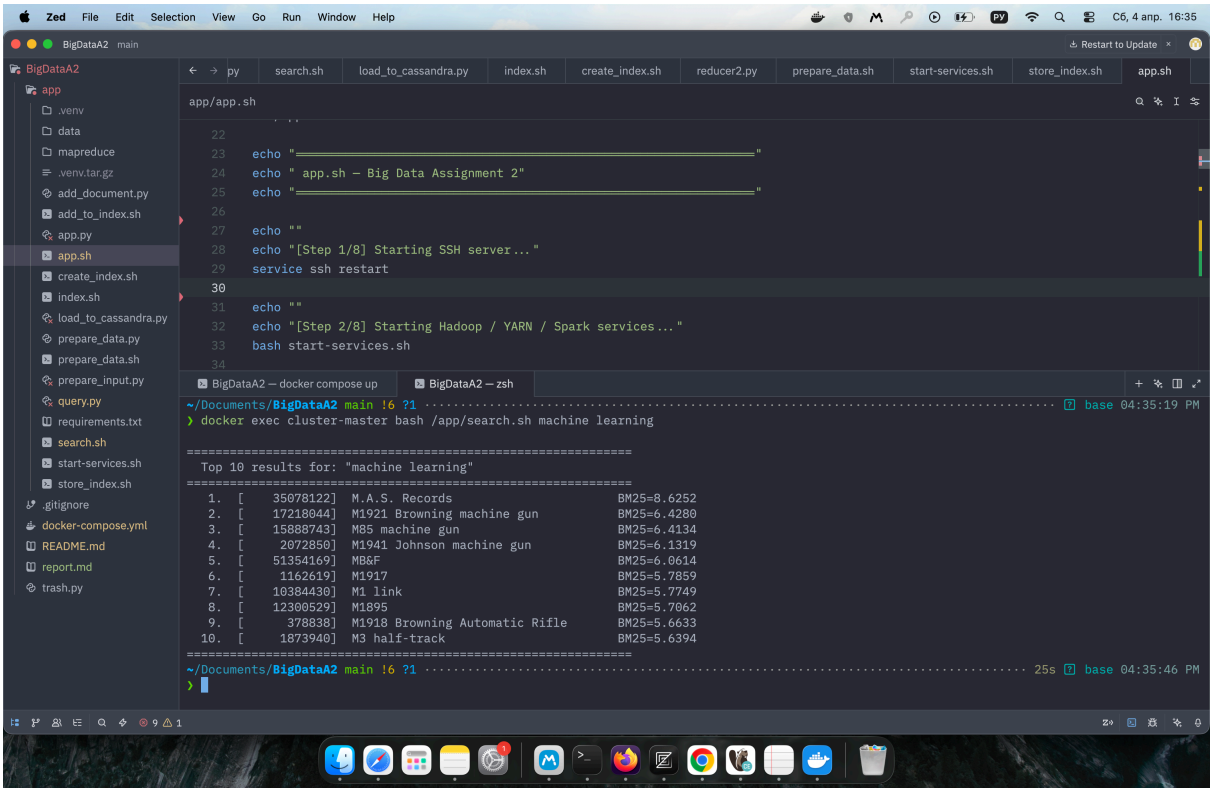
The screenshot below confirms successful indexing:

- HDFS `/data` contains the uploaded documents.
- MapReduce Pipeline 1 wrote `/indexer/index/part-00000`.
- MapReduce Pipeline 2 wrote `/indexer/doc_stats/part-00000`.
- `load_to_cassandra.py` loaded vocabulary, posting list, document statistics, and corpus statistics into Cassandra.

2.3 Search Query Results

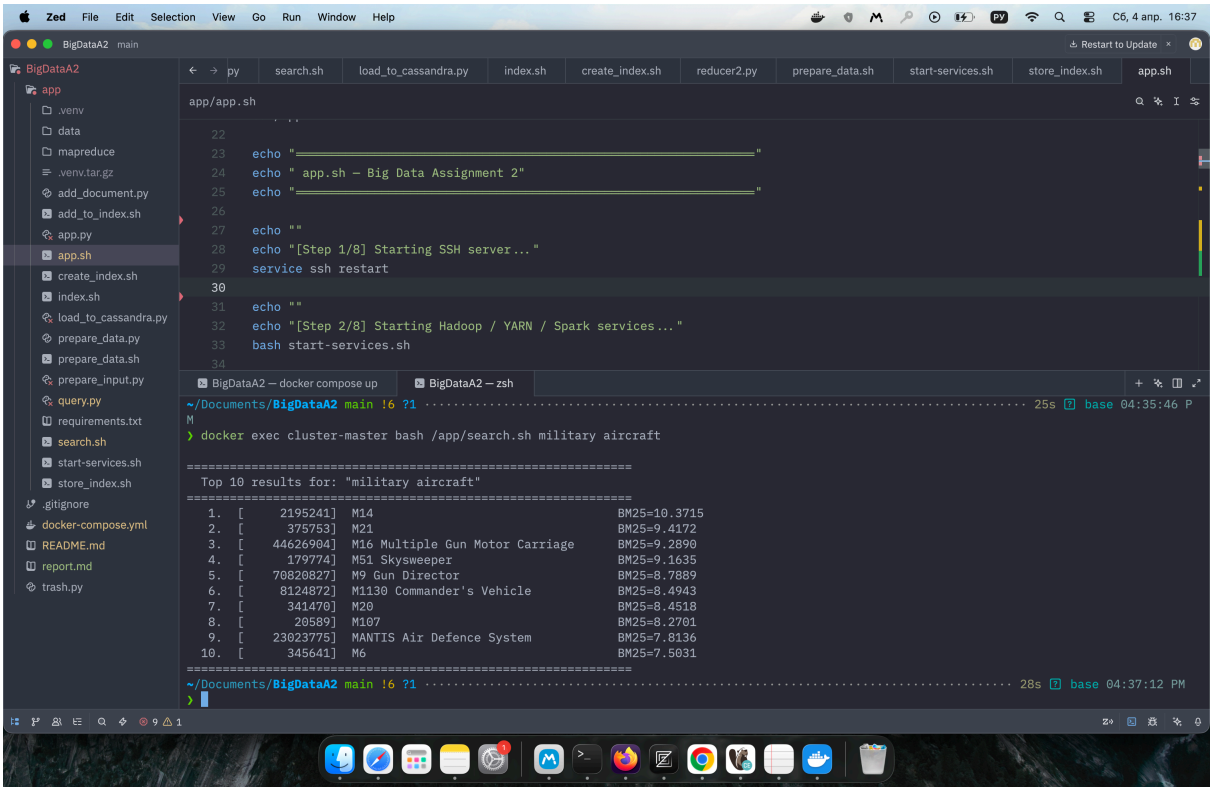
Each query takes approximately 1-3 minutes

Query 1: `"machine learning"`



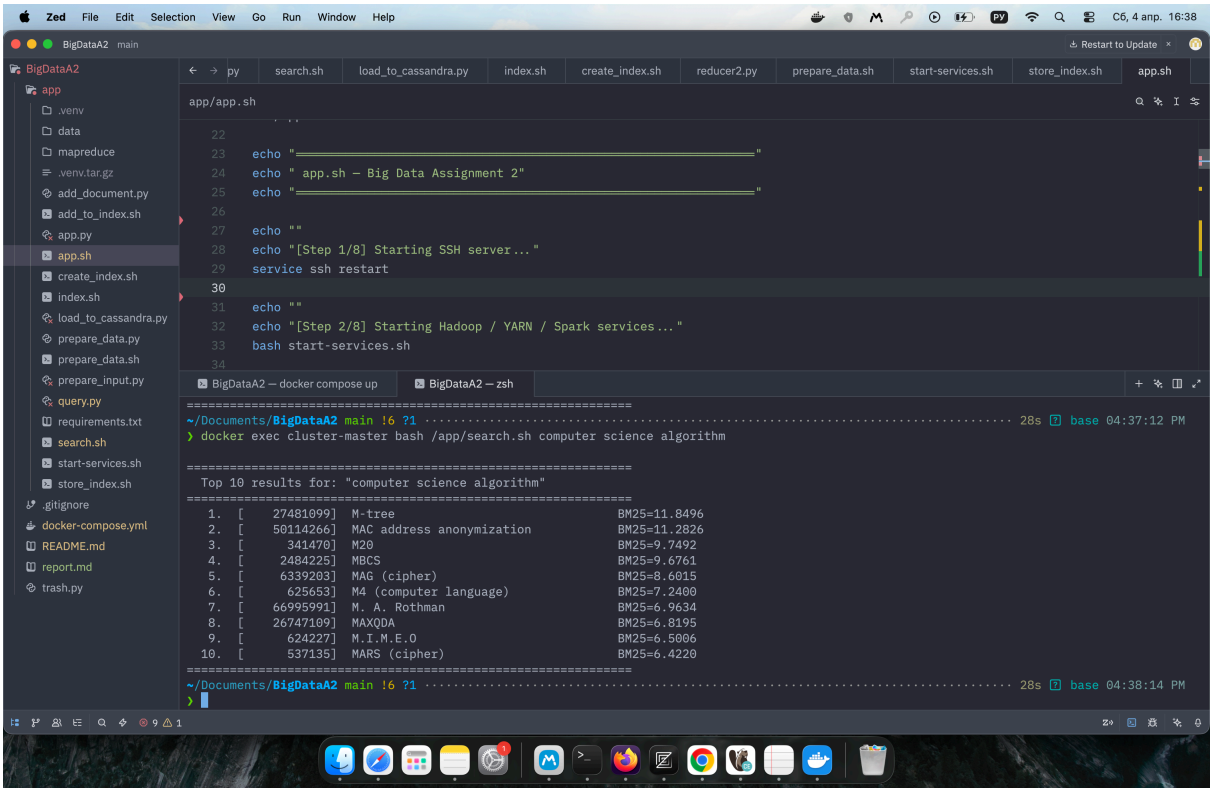
Explanation: BM25 rewards documents where both terms appear frequently relative to document length. Articles specifically about machine learning algorithms or statistical learning methods score highest, while longer articles that mention the topic only in passing are ranked lower.

Query 2: `"military aircraft"`



Explanation: Both terms are domain-specific with moderate document frequency, giving them significant IDF weight. Articles focused on military aviation score well above general articles that mention aircraft or military topics incidentally.

Query 3: `"computer science algorithm"`



Explanation: With three query terms, BM25 scores accumulate from each term independently. "Algorithm" has higher IDF than "computer" or "science" individually, so articles on specific algorithms or computational topics rank highest. Documents matching only one or two terms score noticeably lower.

2.4 Reflections and Findings

- **Short documents can score high** on single-term queries if the query term is dense relative to document length. The parameter `b = 0.75` partially compensates for this; increasing it toward 1.0 would strengthen length normalisation.
- **Common words are naturally suppressed.** Terms like "the" or "and" appear in almost every document, giving them near-zero IDF. No explicit stop-word list is needed.
- **Multi-term queries produce better results.** With three terms the ranking becomes more discriminative, since documents must score well across multiple independent dimensions simultaneously.
- **A small number of documents were excluded** from the index due to non-ASCII characters in their filenames (e.g., articles about M. K. Čiurlionis or İzmir Metro). This is a JVM encoding limitation of the HDFS client in the Docker environment and affects fewer than 1% of the corpus.